Regex Generator from Natural Language

Product Requirements Document (PRD)

Version 0.2 — Updated 16 October 2026
Project status: PLANNING ONLY — coding has not started

1. Document Purpose

This PRD defines the intended user experience, scope, requirements, and acceptance criteria for a tool that converts plain-language descriptions into regular expressions (Regex). It is a planning document, not evidence that implementation has begun.

2. Confirmed Decisions and Status

Item	Current decision	Confidence / status
Development status	Planning only; coding has not started.	Confirmed
AI model approach	Use a cloud API model to help interpret requests and generate Regex.	Confirmed approach; provider/model undecided
Frontend	HTML, CSS, and JavaScript.	Confirmed
Backend framework	Not selected.	Undecided
Cloud model provider / model	Not selected.	Undecided
Regex engine / flavor	Not finalized; must be selected and disclosed to users.	Undecided
Hosting / database / authentication	Not finalized; may not be required for the MVP.	Undecided

3. Product Summary

The product accepts a user's natural-language description (for example, "match a 5-digit postal code"), translates the request into a Regex, validates the expression against the chosen engine, explains its components, and lets the user test sample strings.

4. Problem Statement

People who are unfamiliar with Regex syntax often struggle to translate requirements into correct patterns, understand why a pattern works, and verify whether it matches the intended inputs. The tool aims to reduce that friction while making limitations and validation results visible.

5. Goals

- Generate a candidate Regex from a natural-language requirement using a cloud API model.
- Make assumptions and extracted constraints visible so users can correct ambiguity.
- Validate generated syntax using a deterministic Regex engine before presenting it as valid.
- Explain the pattern in understandable language.
- Allow users to test their own strings and see match / no-match results.
- Avoid claiming a pattern is correct for every situation; communicate the selected Regex flavor and known limitations.

6. Non-Goals for the Initial MVP

- Training or hosting a custom foundation model.
- Guaranteeing that model-generated Regex captures every unstated edge case.
- Supporting every Regex flavor on day one.

- User accounts, cloud history, team collaboration, or paid subscriptions unless later approved.
- Executing arbitrary user-provided code.

7. Target Users

- Students learning Regex and programming fundamentals.
- Developers who need a quick first draft of a pattern.
- QA / testing users who need to validate string formats.
- Non-experts who can describe a text-matching requirement more easily than writing Regex.

8. MVP Functional Requirements

ID	Requirement	Priority	Acceptance criteria
FR-01	Natural-language input	Must	User can enter a requirement in a text field and submit it.
FR-02	Requirement / constraint extraction	Must	System identifies key constraints and shows assumptions or asks for clarification when the request is ambiguous.
FR-03	Cloud API generation	Must	Backend calls the selected cloud model securely; API secrets are not exposed in browser code.
FR-04	Deterministic validation	Must	Candidate pattern is checked by the selected Regex engine; invalid syntax is not labelled valid.
FR-05	Regex display and copy	Must	User can view and copy the generated pattern.
FR-06	Explanation	Must	System provides a plain-language explanation tied to the actual pattern.
FR-07	Test strings	Must	User can enter sample strings and see match / no-match results using the same engine.
FR-08	Error and uncertainty handling	Must	API failures, ambiguous requests, and validation errors are shown clearly.
FR-09	Regex flavor disclosure	Must	UI identifies the engine/flavor used, once chosen.
FR-10	Match highlighting	Should	Matched spans are highlighted where supported by the selected engine.
FR-11	Refinement	Could	User can request adjustments such as case-insensitive matching or optional separators.

9. Non-Functional Requirements

- Security: keep cloud API credentials on the server; validate request sizes and inputs; avoid logging secrets or unnecessary personal data.
- Reliability: model output must be treated as untrusted candidate content and validated before use.
- Usability: keep the flow understandable for beginners and provide actionable error messages.
- Performance: show loading and timeout/error states; target latency and limits are to be defined after provider selection.
- Accessibility: use semantic HTML, keyboard-operable controls, labels, and readable contrast.
- Maintainability: separate UI, API orchestration, validation, and explanation responsibilities.

10. High-Level User Flow

1. User enters a natural-language requirement.
2. System checks whether the request is sufficiently clear and extracts key constraints.
3. Backend sends a constrained prompt to the chosen cloud model.
4. Backend parses the response and validates the Regex with the selected engine.
5. If validation fails, the system may retry safely or return an error; it must not silently present invalid output as valid.
6. UI shows the Regex, flavor, explanation, assumptions, and validation status.
7. User tests sample strings and optionally refines the requirement.

11. Success Measures (Targets to be Set)

- Task success rate on a curated set of representative requirements.
- Percentage of generated patterns that pass engine syntax validation.
- Match-result correctness against hand-authored test cases.
- User ability to understand and edit the result in usability testing.
- Latency, API cost per request, and error rate after a provider is selected.

Numeric targets are intentionally not fixed yet; they should be agreed after a test set, model, and Regex engine are selected.

12. Risks and Mitigations

Risk	Mitigation
Ambiguous natural language	Show extracted constraints and assumptions; ask clarifying questions.
Plausible but incorrect Regex	Use deterministic validation and curated positive/negative test cases.
Model hallucination or malformed output	Use a structured response contract, parsing, retries with limits, and safe failure.
Regex denial-of-service / catastrophic backtracking	Choose safe limits and consider a timeout-capable or safer engine; test adversarial patterns.
Cloud API cost, latency, or outage	Add timeouts, usage limits, observability, and a provider contingency plan.
Secrets exposed in frontend	Keep API calls and secrets on the backend; never embed secret keys in JavaScript.

13. Open Decisions

- Which cloud API provider and model will be used?
- Which backend language/framework will call the provider and validate requests?
- Which Regex engine/flavor is the initial supported target?
- Should the MVP support one flavor only or multiple explicitly selected flavors?
- Will test strings be processed only in the browser/backend transiently, or stored?
- Where will the app be hosted, and what budget / usage limits apply?
- What dataset and evaluation rubric will be used to measure quality?

14. Release / Readiness Checklist

- Confirm cloud model provider and API terms.
- Choose Regex engine and document its flavor-specific syntax.
- Implement server-side credential handling.
- Build a representative test suite with positive, negative, boundary, and malformed cases.
- Test ambiguous requests, API failures, timeouts, and unsafe patterns.
- Conduct accessibility and beginner usability checks.

Document status: planning draft. No coding or implementation is claimed to have started.